

# SMACP : Système de Monitoring Avancé d'une Centrale Photovoltaïque

Etudiant 3 – Eliott MOLARD

## Objectifs étudiant 3

- Tableau de bord administrateur / Chercheur / Utilisateur
- Infrastructure réseau LAN / IOT
- Intégration des solutions E1, E2
- Cyber sécurisation de la solution

Mes objectifs au sein du projet SMACP étaient de participer à la maintenance et à l'évolution de l'infrastructure de supervision de la centrale photovoltaïque. Mon travail s'est principalement concentré sur l'application web Django, l'administration des serveurs ainsi que l'exploitation des données collectées par les différents équipements du système.

Dans ce cadre, mes objectifs étaient les suivants :

- Assurer la reprise en main de l'environnement de développement et de l'infrastructure serveur,
- Participer à l'amélioration du tableau de bord destiné aux différents utilisateurs de la plateforme (administrateur, chercheur et utilisateur),
- Garantir (répliquer) le bon fonctionnement de la base de données et des services associés à l'application web,
- Intégrer les données des différentes batteries au sein des interfaces de visualisation,
- Étudier l'architecture de communication IoT existante ainsi que les solutions MQTT utilisées pour la transmission des données,
- Contribuer à la sécurisation et à la fiabilité de la solution à travers la gestion des accès, la sauvegarde des données et l'utilisation d'environnements de test indépendants.

Ces différents objectifs avaient pour finalités d'améliorer la supervision de la centrale photovoltaïque tout en préparant les futures évolutions de la plateforme.

## 1) Diagrammes et Technologies

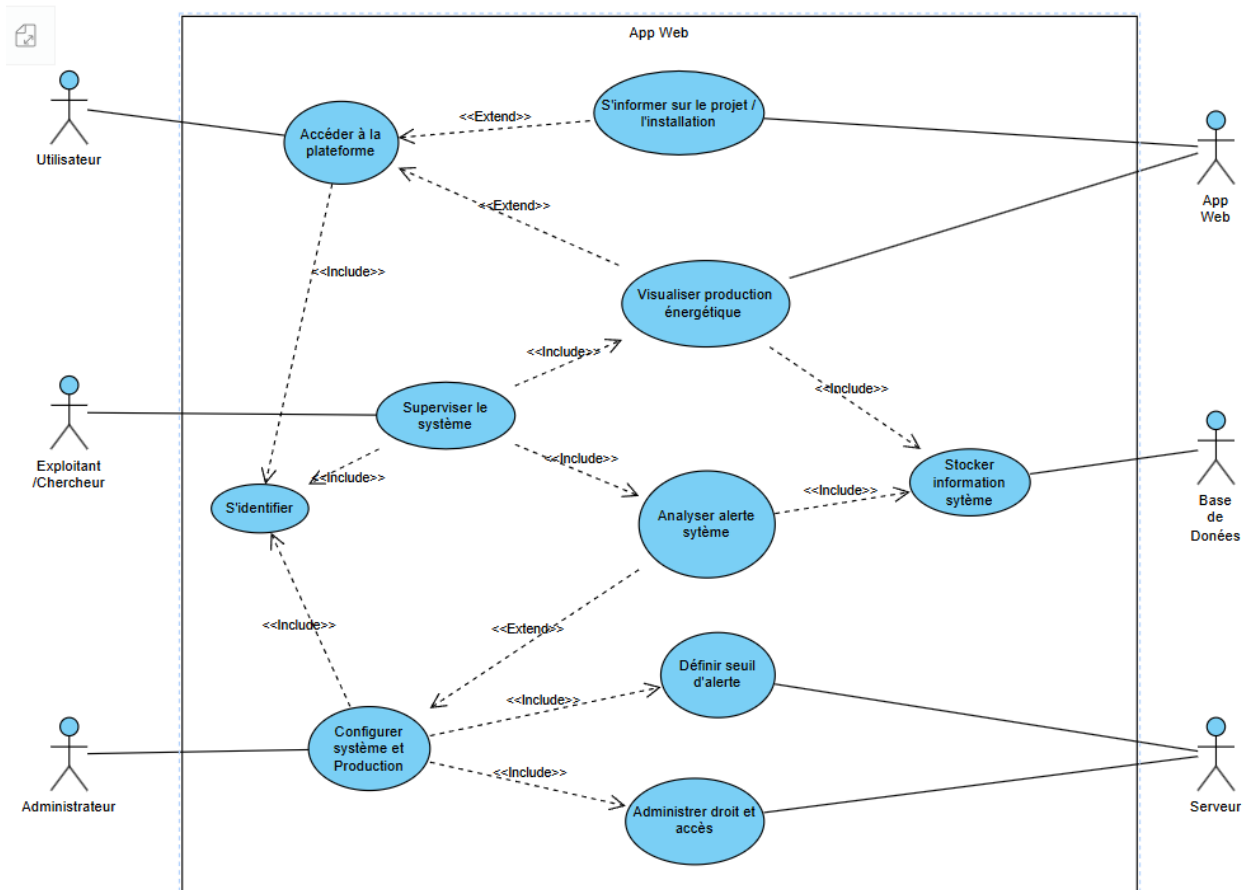


Figure 1 : Cas d'utilisation Personnel

| Technologie | Utilisation   |
|-------------|---------------|
| Python      | langage       |
| Django      | site web      |
| PostgreSQL  | base          |
| Ubuntu      | serveur       |
| GitHub      | versionnement |
| VSCoDe SSH  | accès distant |
| FileZilla   | transfert     |

Figure 2 : Technologie utilisé

### Etudiant 3 – Eliott MOLARD

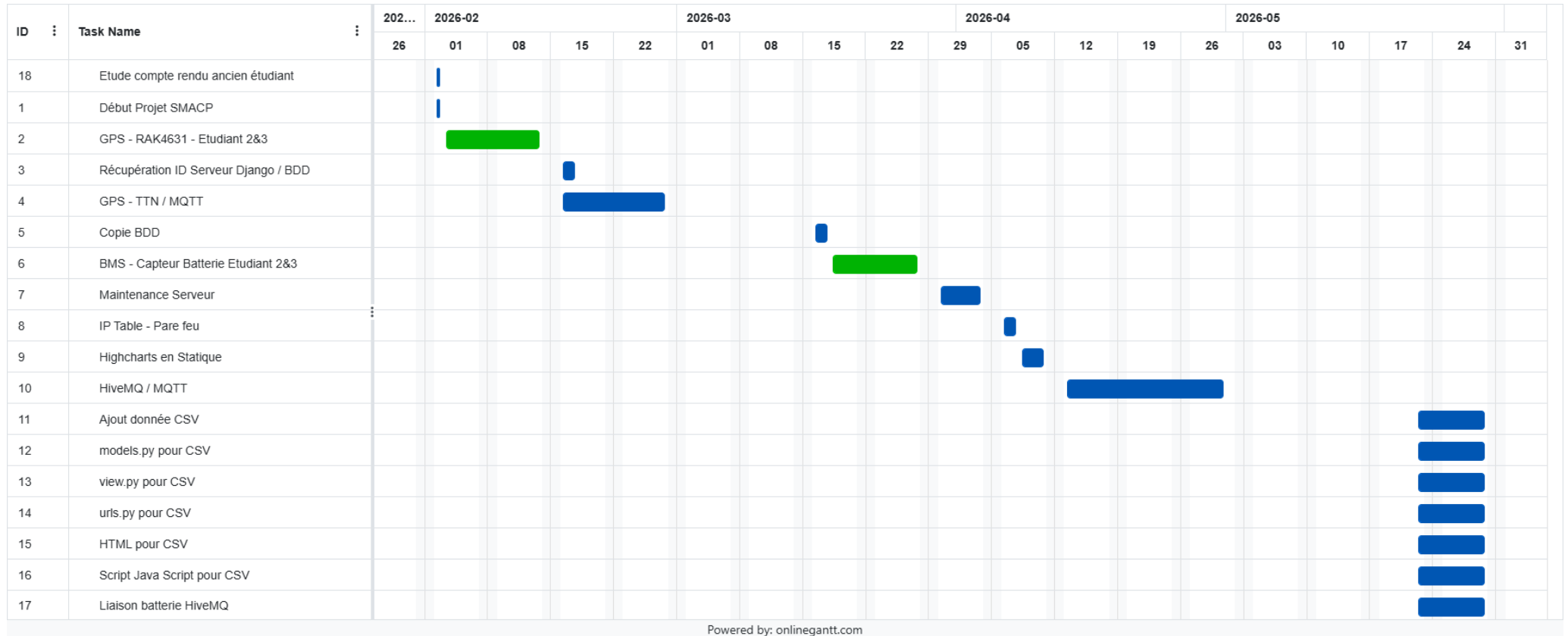


Figure 3 : Diagramme de Gantt des tâches réalisées

Début du projet le 1 février.

En bleu les taches personnelles

En vert les taches communes réaliser avec les étudiants de mon groupe

Etudiant 1 : Dijoux Mathias

Etudiant 2 : Willan Banor

Etudiant 3 : Eliott Molar

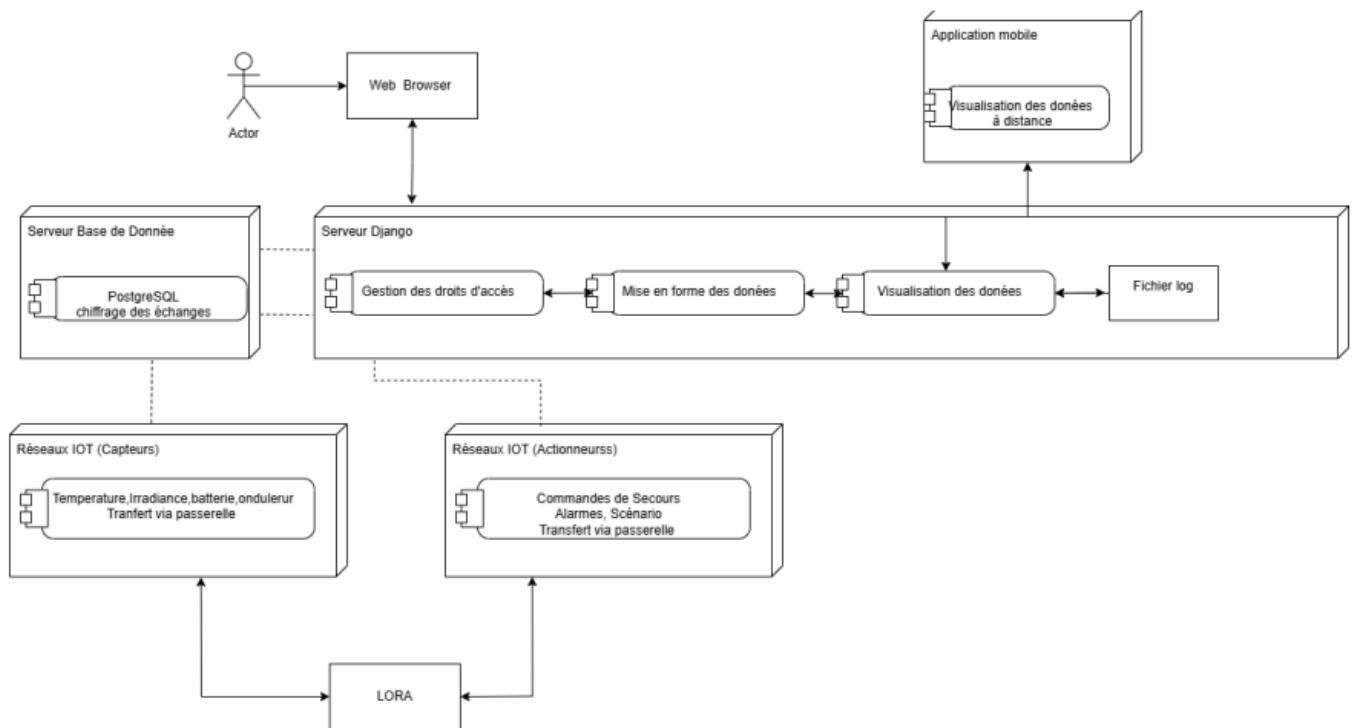


Figure 4 : Diagramme de déploiement

## 2) Prise en main de l'environnement et Git Hub

### 2.1 VS Code & SSH

Pour la reprise en main du projet la récupération des différents identifiants était nécessaire pour accéder au serveur Django (site WEB) et BDD depuis une connexion SSH effectuée avec l'aide de Visual Studio Code qui sera l'environnement principal de travail. VS Code SSH permet d'avoir une interface graphique plus lisible notamment pour la navigation dans les dossiers et fichiers.

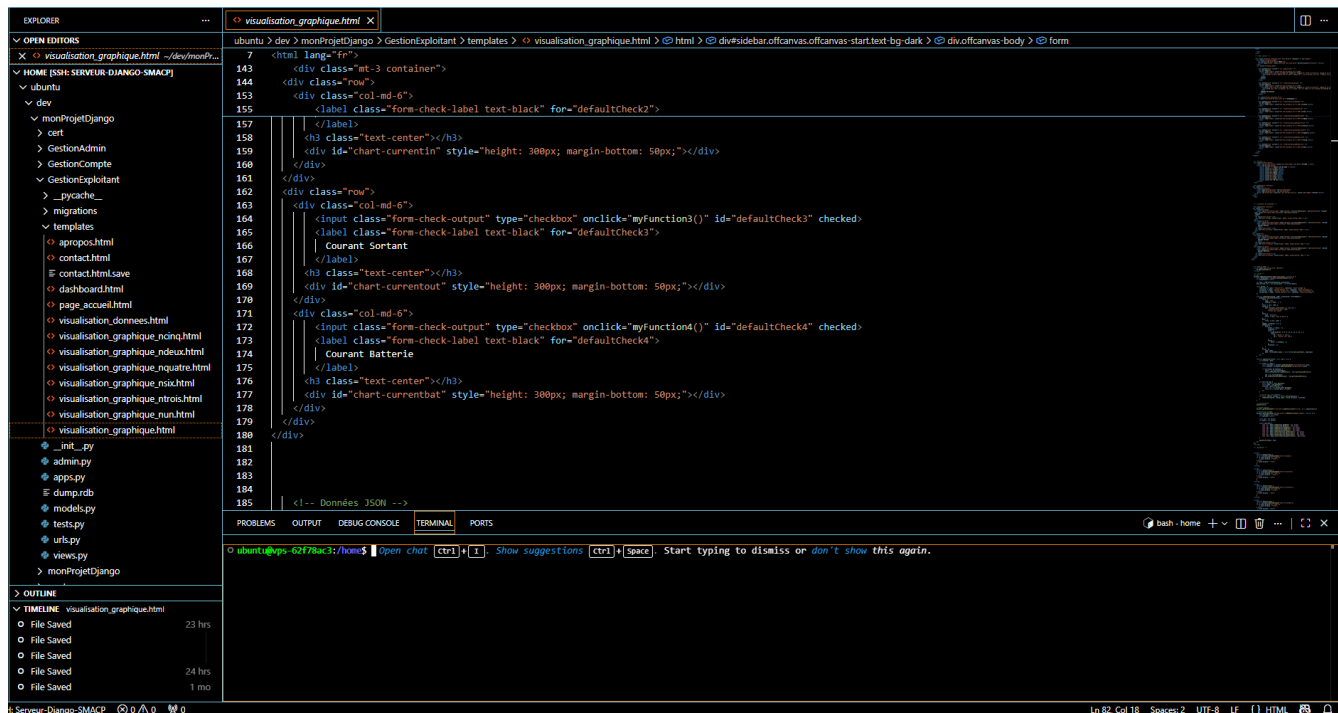


Figure 5 : Exemple d'utilisation VS Code en SSH (Serveur Django)

## 2.2 GitHub

Dans le cadre du projet, GitHub a été utilisé pour assurer le versionnement du code et de la documentation. Cet outil a permis de conserver un historique des modifications, de faciliter le travail collaboratif et de garantir la traçabilité des évolutions apportées à l'application.

### GitHub : Gestion de version et collaboration

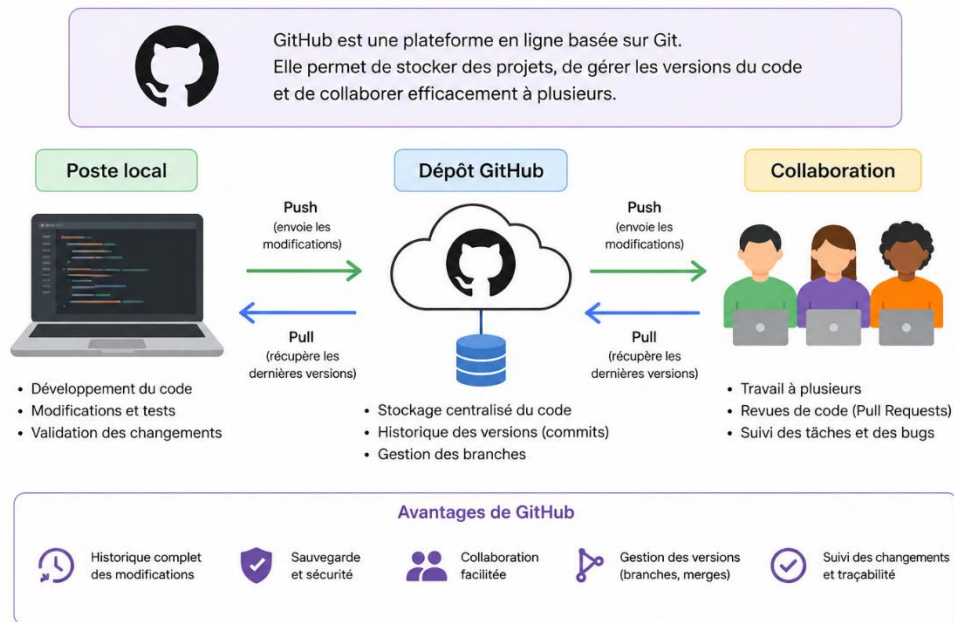


Figure 6 : Présentation GitHub

Dépôt code : <https://github.com/Eliott-07/Centrale-PV-Securisee-2026-Code.git>

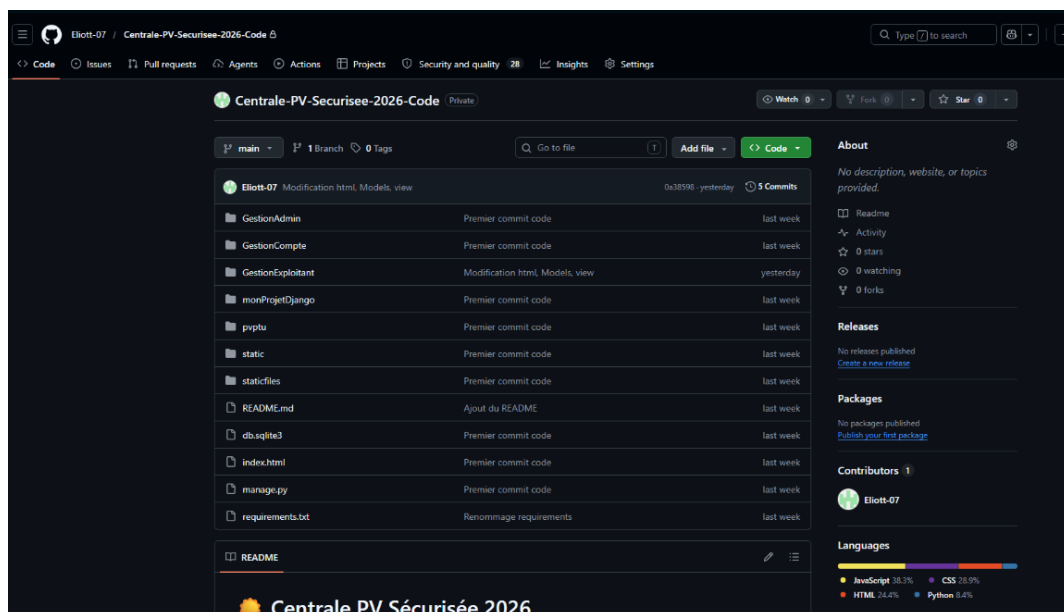


Figure 7 : Dépôt Code

Dépôt doc : <https://github.com/Eliott-07/Centrale-PV-Securisee-2026.git>

### 3) Cybersécurité du projet

#### 3.1 Considérations de cybersécurité :

La sécurisation de la plateforme constitue un enjeu important du projet. Les données collectées et stockées au sein de l'application doivent être protégées contre les accès non autorisés afin de garantir leur intégrité et leur confidentialité.

Dans le cadre du projet, une attention particulière a été portée à la gestion des accès au serveur via SSH ainsi qu'à la protection de la base de données PostgreSQL. L'utilisation d'environnements de test distincts de la production permet également de limiter les risques lors des phases de développement et de validation.

Pour la partie production l'infrastructure Django ainsi que la Base de données principale sont stockés sur deux serveurs distants chez l'entreprise française OVH pour des raisons de souveraineté (sécurité et de protection des données).

Les recommandations publiées par l'ANSSI constituent une référence pour la mise en œuvre de bonnes pratiques de sécurité concernant l'administration des serveurs, la gestion des accès et le maintien en condition de sécurité des systèmes d'information.

#### 3.2 Certificats de sécurité :

Dans le cadre du projet SMACP, la sécurisation des communications constitue un élément important afin de garantir la confidentialité et l'intégrité des données échangées entre les différents équipements et les utilisateurs de la plateforme.

Pour cela, les serveurs web utilisent généralement des certificats TLS (Transport Layer Security). Ces certificats permettent de mettre en œuvre le protocole HTTPS, qui assure le chiffrement des données transitant sur le réseau. Ainsi, les informations échangées entre le navigateur de l'utilisateur et le serveur Django ne peuvent pas être lues ou modifiées facilement par un tiers malveillant.

Les certificats modernes reposent notamment sur la famille d'algorithmes SHA-2 (Secure Hash Algorithm 2), reconnue pour son niveau de sécurité élevé. Contrairement aux anciens algorithmes tels que SHA-1, aujourd'hui considérés comme vulnérables, SHA-2 offre une meilleure résistance aux attaques cryptographiques et est largement utilisé dans les certificats TLS actuels.

Les algorithmes SHA-2 sont également utilisés pour générer des empreintes numériques (hash) permettant de vérifier l'intégrité des données. Parmi eux, SHA-256 est l'un des plus répandus.

Lorsqu'un utilisateur crée un mot de passe, celui-ci n'est généralement pas stocké en clair dans la base de données. Une fonction de hachage est appliquée afin de transformer le mot de passe en une empreinte numérique difficilement réversible. Cette méthode permet de limiter les risques en cas de compromission de la base de données.

Cependant, même si SHA-256 demeure un algorithme robuste, l'utilisation de mots de passe faibles peut faciliter certaines attaques par dictionnaire ou par force brute. Il est donc recommandé d'utiliser des mots de passe complexes et de mettre en œuvre des mécanismes complémentaires tels que le salage (salt) et les algorithmes spécialisés de stockage des mots de passe.

Si dessous vous avez un schéma explicatif de la sécurisation des échanges et des données.

## Schéma de sécurisation des échanges et des données

Projet SMACP



Figure 8 : Schéma de sécurisation des échanges et des données

### 3.3 CVE et CVSS :

Dans le cadre du maintien en condition de sécurité, il est recommandé d'effectuer une veille régulière concernant les vulnérabilités publiées sous forme de CVE (Common Vulnerabilities and Exposures). Le score CVSS associé à ces vulnérabilités permet d'évaluer leur niveau de criticité et d'identifier les correctifs devant être appliqués en priorité.

### 4) Accès et copie de la Base de Donnée

Une partie importante du projet consistait à récupérer une copie locale de la base de données PostgreSQL afin de mettre en place un environnement de test indépendant du serveur principal.

Plusieurs difficultés ont été rencontrées lors de cette étape.

Dans un premier temps, les informations de connexion à PostgreSQL étaient incomplètes. Après plusieurs recherches dans les documents techniques disponibles, les identifiants nécessaires ont finalement été retrouvés.

Par la suite, un problème de permissions empêchait la restauration de la base de données sur la machine de test. Afin de résoudre ce problème, le fichier de configuration `pg_hba.conf` a été modifié.

Ce fichier permet de gérer les méthodes d'authentification et les autorisations PostgreSQL. La méthode `md5`, qui demande un mot de passe, a temporairement été remplacée par `trust` dans un environnement sécurisé afin de faciliter les opérations de restauration.



*Représentation du fichier (pas le fichier réel !)*

```
# -----
#
# If you want to allow non-local connections, you need to add more
# "host" records. In that case you will also need to make PostgreSQL
# listen on a non-local interface via the listen_addresses
# configuration parameter, or via the -i or -h command line switches.

# TYPE DATABASE USER ADDRESS METHOD
# "local" is for Unix domain socket connections only
local all all trust
# IPv4 local connections:
host all all 127.0.0.1/32 md5
host all all 192.168.0.0/16 scram-sha-256
# IPv6 local connections:
host all all ::1/128 trust
# Allow replication connections from localhost, by a user with the
# replication privilege.
local replication all scram-sha-256
host replication all 127.0.0.1/32 scram-sha-256
host replication all ::1/128 scram-sha-256
```

*Figure 9 : Fichier pg\_hba.conf*

Sous Linux le fichier se trouve généralement dans les dossiers suivants :

*etc/postgresql/15/main/pg\_hba.conf*

(A noter que le numéro dans le chemin d'accès peut changer en fonction de votre version de PostgreSQL)

Afin de pouvoir réaliser des tests sans impacter l'environnement de production, une copie locale de la base de données PostgreSQL a été mise en place. Cette étape était nécessaire pour disposer d'un environnement de développement indépendant et sécurisé.

La récupération de cette base de données a nécessité plusieurs opérations préalables, notamment la recherche des informations de connexion ainsi que la vérification des droits d'accès au serveur PostgreSQL. Plusieurs difficultés ont été rencontrées, notamment liées aux permissions utilisateur et aux paramètres d'authentification de la base de données.

Des ajustements de configuration ont été effectués afin de permettre la restauration de la sauvegarde sur une machine de test. Une incompatibilité entre les versions PostgreSQL du serveur et de l'environnement local a également nécessité des vérifications complémentaires avant de pouvoir finaliser l'importation de la base.

Une fois ces différents problèmes résolus, la base de données a pu être restaurée avec succès sur l'environnement de test. Cette copie locale a ensuite servi de support pour l'ensemble des développements et des essais réalisés durant le projet.



## 5) Copie du Projet Django

### 5.1 Erreur Git push

Pour la copie du code source GitHub devait servir de versioning entre le serveur et le poste local afin d'apporter les modifications nécessaires au code mais à cause d'une mauvaise configuration du fichier .gitignore et d'une mauvaise commande « git add . » plusieurs fichiers et dossiers ont donc disparu notamment un fichier wsgi qui permet la configuration de Gunicorn et donc au bon fonctionnement du site web

```
ubuntu@vps-62f78ac3:~$ sudo systemctl status gunicorn
* gunicorn.service - gunicorn daemon
   Loaded: loaded (/etc/systemd/system/gunicorn.service; enabled; preset: enabled)
   Active: failed (Result: exit-code) since Wed 2026-04-01 19:32:32 UTC; 14h ago
     Duration: 135ms
   Invocation: 18a714d499f54382bcf817927be0a491
   TriggeredBy: * gunicorn.socket
   Process: 1385 ExecStart=/home/ubuntu/dev/envdjango/bin/gunicorn --access-logfile - --workers 3 --bind unix:/run/gunicorn.sock monProjetDjango.wsgi:application (code=exited, status=3)
   Main PID: 1385 (code=exited, status=3)

Apr 01 19:32:31 vps-62f78ac3 gunicorn[1386]: ModuleNotFoundError: No module named 'monProjetDjango.wsgi'
Apr 01 19:32:31 vps-62f78ac3 gunicorn[1386]: [2026-04-01 19:32:31 +0000] [1386] [INFO] Worker exiting (pid: 1386)
Apr 01 19:32:31 vps-62f78ac3 gunicorn[1385]: [2026-04-01 19:32:31 +0000] [1385] [ERROR] Worker (pid:1386) exited with code 3
Apr 01 19:32:31 vps-62f78ac3 gunicorn[1385]: [2026-04-01 19:32:31 +0000] [1385] [ERROR] Shutting down: Master
Apr 01 19:32:31 vps-62f78ac3 gunicorn[1385]: [2026-04-01 19:32:31 +0000] [1385] [ERROR] Reason: Worker failed to boot.
Apr 01 19:32:32 vps-62f78ac3 systemd[1]: gunicorn.service: Main process exited, code=exited, status=3/NOTIMPLEMENTED
Apr 01 19:32:32 vps-62f78ac3 systemd[1]: gunicorn.service: Failed with result 'exit-code'.
Apr 01 19:32:32 vps-62f78ac3 systemd[1]: gunicorn.service: Start request repeated too quickly.
Apr 01 19:32:32 vps-62f78ac3 systemd[1]: gunicorn.service: Failed with result 'exit-code'.
Apr 01 19:32:32 vps-62f78ac3 systemd[1]: Failed to start gunicorn.service - gunicorn daemon.
```

Figure 10 : Erreur Gunicorn

Une erreur 502 Bad Gateway est ensuite afficher sur le site puisque Nginx ne peut plus communiquer avec Gunicorn.



Figure 11 : Erreur 502 Bad Gateway

Pour corriger cette erreur une restauration complète du serveur a été réalisée afin de récupérer les fichiers et dossiers perdu enfin Filezilla a été utilisé afin de transférer les dossiers et fichiers du code.

## 6) Modifications Codes HTML

### 6.1 Highcharts CDN & Erreur

Le site Web utilise des liens Highcharts que l'on appelle des liens CDN (Content Delivery Network) mais ces liens présentent plusieurs problèmes :

- Ils proviennent de serveur externe donc dépendance
- Ils peuvent être limités dans le temps et donc bloquer
- Ils peuvent être bloqués s'il y a trop de requête

Voici une représentation de ces liens CDN dans un code HTML :

```

8      <meta charset="UTF-8">
9      <title>Dashboard</title>
10     <link rel="stylesheet" href="{% static 'css/styles.css' %}">
11     <script src="https://code.highcharts.com/highcharts.js"></script>
12     <script src="https://code.highcharts.com/modules/exporting.js"></script>
13     <script src="https://code.highcharts.com/modules/export-data.js"></script>
14     <script src="https://code.highcharts.com/modules/accessibility.js"></script>
15     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
16 </head>
17

```

Figure 12 : Liens CDN Highcharts

Les liens étaient devenus invalide / bloquer par highcharts et rendait donc impossible l'apparition de nos graphes sur le site :

The screenshot shows a web application interface on the left and a browser's Network tab on the right. The web application has a header 'Graphique des données' and a 'Menu' button. Below the header, there is a 'Période' dropdown menu with options like 'jj/mm/aaaa --:--' and a 'Filtrer' button. The browser's Network tab shows a list of requests, including 'dashboard/', 'isolated-firstjit', 'bootstrap.min.css', 'bootstrap.bundle.min.js', 'styles.css', 'highcharts.js', 'exporting.js', 'export-data.js', 'accessibility.js', 'bootstrap.min.css', 'dataimage/svg+xml...', 'dataimage/svg+xml...', and 'favicon.ico'. The 'favicon.ico' request is highlighted, showing a 404 status code and the message 'Not Found'.

Figure 13 : Graphe inactif à cause des liens CDN

## 6.2 Highcharts en statique

Pour résoudre ce problème il faut donc remplacer ces liens CDN par des liens statiques qui redirigent vers les ressources qui ont été téléchargées puis intégrer dans les dossiers statiques de Django.

```

<head>
  <meta charset="UTF-8">
  <title>Dashboard</title>
  <link rel="stylesheet" href="{% static 'css/styles.css' %}">
  <script src="{% static 'js/highcharts.js' %}"></script>
  <script src="{% static 'js/exporting.js' %}"></script>
  <script src="{% static 'js/export-data.js' %}"></script>
  <script src="{% static 'js/accessibility.js' %}"></script>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

```

Figure 14 : Liens Highcharts en statique

La page WEB re affiche donc les graphes de donnée :

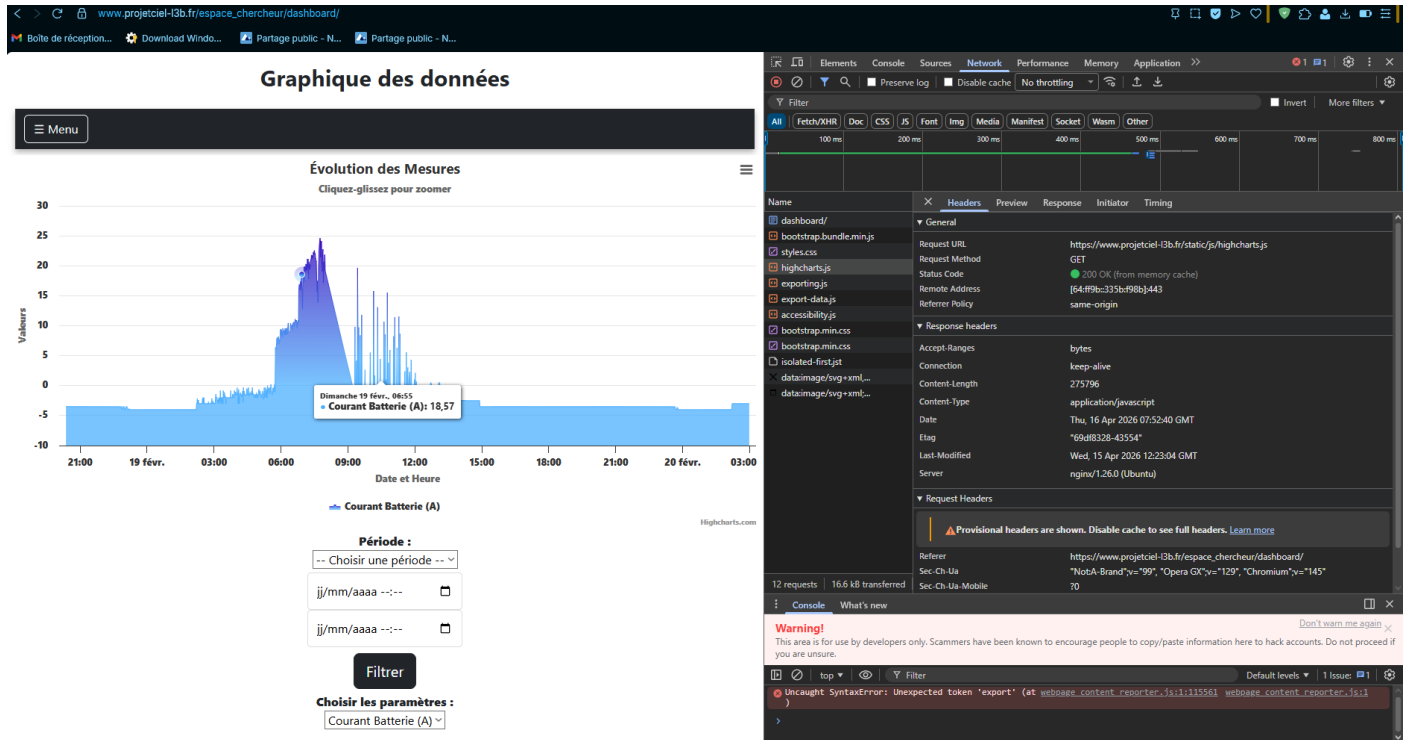


Figure 15 : Page avec graphe liens en statique

## 7) Ajout des données CSV de la Batterie et affichage Web

L'ajout des données CSV de la batterie avait déjà été réalisé par les anciens élèves travaillant sur le projet, ces données correspondent à celles fournies par le module 123 Smart BMS qui permet de récupérer les différentes données des différentes cellules de la batterie. La date des données proviennent de la période 2019 à 2023 quand l'installation fonctionnait correctement.



Figure 16 : Batterie

Les données récupérées sont montrées dans ce document fourni par 123 Smart BMS

| Address: | Byte 0                 | Byte 1 | Byte 2           | Byte 3               | Byte 4                | Byte 5       | Byte 6               | Byte 7 |
|----------|------------------------|--------|------------------|----------------------|-----------------------|--------------|----------------------|--------|
| N+0      | Total voltage          |        | Current IN       |                      | Current OUT           |              | Current Battery      |        |
| N+1      | Energy stored          |        |                  |                      | Battery capacity      |              | SOC                  | -      |
| N+2      | Energy today collected |        |                  |                      | Energy today consumed |              |                      |        |
| N+3      | Total energy collected |        |                  |                      | Total energy consumed |              |                      |        |
| N+4      | Cell voltage MIN       |        | Cell voltage MAX |                      | Cell voltage Bypass   |              | -                    | -      |
| N+5      | Cell voltage Lowest    |        | Low Nr           | Cell voltage Highest |                       | High Nr      | Sbyte 1              | Sbyte2 |
| N+6      | Tmp,Lowest             | Low Nr | Tmp,Highest      | High Nr              | Min charg temp        | Min dis temp | Max temp             | -      |
| N+7      | Current cell voltage   |        | Cur.Temp         | Cur. Nr              | Cell cnt              | -            | Isolation resistance |        |

Figure 17 : BUS CAN Data

A la reprise du projet seule la donnée N+0 de la batterie était présentée sur le site Web (Figure les autres bien qu'ils étaient déjà présents dans la BDD et les models django n'avaient pas été ajoutées à temps par les anciens élèves, l'objectif était donc d'adapter le système existant afin de permettre l'affichage des données manquante (N+1, N+2 etc...)

Pour cela, j'ai d'abord analysé le fonctionnement déjà en place pour la batterie N+0. Cette étape m'a permis de comprendre :

- La récupération des données depuis la base de données
- Le fonctionnement des vues Django avec le fichier view.py
- Le fonctionnement des modèles Django avec le fichier models.py
- Le fonctionnement des URLs Django avec le fichier django.py
- Le traitement des données dans les scripts JavaScript
- L'affichage des graphiques avec Highcharts.

Après cette phase d'analyse, plusieurs modifications ont été réalisées dans différents fichiers du projet :

- models.py
- views.py
- urls.py
- Fichiers HTML
- Scripts JavaScript

Le fichier models.py a été vérifié afin de s'assurer que les modèles correspondant aux différentes données de batteries étaient correctement définis et exploitables par l'application (en rapport avec les données stockées dans la Base de données).

```

class Donnee(models.Model):
    date = models.DateTimeField(primary_key=True)
    total_volt = models.FloatField()
    currentin = models.FloatField()
    currentout = models.FloatField()
    current_bat = models.FloatField()

    class Meta:
        managed = False
        db_table = 'current'

class Nzero(models.Model):
    date = models.DateTimeField(primary_key = True)
    total_volt = models.FloatField()
    currentin = models.FloatField()
    currentout = models.FloatField()
    current_bat = models.FloatField()
    class Meta:
        managed = False
        db_table = 'DB_nzero'

```

Figure 18 : models.py N+0, N+1

Le fichier views.py a ensuite été adapté afin de récupérer les nouvelles données depuis la base de données et de les transmettre aux pages web. Cette étape permet d'assurer la communication entre les données stockées dans PostgreSQL et l'interface utilisateur.

```

43 def GrapheDesDonnees(request):
44     data = list(Donnee.objects.values('date', 'total_volt', 'currentin', 'currentout', 'current_bat'))
45     data_json = json.dumps(data, default=str) # Convertir QuerySet en JSON
46     return render(request, "visualisation_graphique.html", {'data_json': data_json})
47
48 def GrapheDesDonnees_nun(request):
49     data = list(Nun.objects.values('date', 'Energy_stored_Wh', 'Battery_capacity_kWh', 'SOC_percent', 'nn'))
50     data_json = json.dumps(data, default=str)
51     return render(request, "visualisation_graphique_nun.html", {'data_json': data_json})
52
53 def GrapheDesDonnees_ndeux(request):
54     data = list(Ndeux.objects.values('date', 'Energy_today_collected_kWh', 'Energy_today_consummed_kWh'))
55     data_json = json.dumps(data, default=str) # Convertir QuerySet en JSON
56     return render(request, "visualisation_graphique_ndeux.html", {'data_json': data_json})
57
58 def GrapheDesDonnees_ntrois(request):
59     data = list(Ntrois.objects.values('date', 'Total_Energy_today_collected_kWh', 'Total_Energy_today_consummed_kWh'))
60     data_json = json.dumps(data, default=str)
61     return render(request, "visualisation_graphique_ntrois.html", {'data_json': data_json})
62
63 def GrapheDesDonnees_nquatre(request):
64     data = list(Nquatre.objects.values('date', 'Cell_voltage_min_V', 'Cell_voltage_max_V', 'Cell_voltage_bypass_V', 'nonnommeeun', 'nonnommeedeux', 'nonnommeetrois'))
65     data_json = json.dumps(data, default=str)
66     return render(request, "visualisation_graphique_nquatre.html", {'data_json': data_json})
67
68 def GrapheDesDonnees_ncinq(request):
69     data = list(Ncinq.objects.values('date', 'Cell_voltage_lowest_V', 'Low_nr', 'Cell_voltage_highest_V', 'High_nr', 'Sbyteun', 'Sbytedeux'))
70     data_json = json.dumps(data, default=str)
71     return render(request, "visualisation_graphique_ncinq.html", {'data_json': data_json})
72
73 def GrapheDesDonnees_nsix(request):
74     data = list(Nsix.objects.values('date', 'Tmp_lowest_deg', 'Tmp_highest_deg', 'Min_charg_temp_deg', 'Max_temp_deg'))
75     data_json = json.dumps(data, default=str) # Convertir QuerySet en JSON
76     return render(request, "visualisation_graphique_nsix.html", {'data_json': data_json})

```

Figure 19 : views.py N+0 à N+6

Ensuite, le fichier urls.py a été modifié pour rajouter les liens et donc l'accès aux pages permettant la visualisation des différentes données.

```

#ajout 2026
path('grapheDesDonnees_nun/', views.GrapheDesDonnees_nun, name='visualisation_graphique_nun'),
path('grapheDesDonnees_ndeux/', views.GrapheDesDonnees_ndeux, name='visualisation_graphique_ndeux'),
path('grapheDesDonnees_ntrois/', views.GrapheDesDonnees_ntrois, name='visualisation_graphique_ntrois'),
path('grapheDesDonnees_nquatre/', views.GrapheDesDonnees_nquatre, name='visualisation_graphique_nquatre'),
path('grapheDesDonnees_ncinq/', views.GrapheDesDonnees_ncinq, name='visualisation_graphique_ncinq'),
path('grapheDesDonnees_nsix/', views.GrapheDesDonnees_nsix, name='visualisation_graphique_nsix'),
]

```

Figure 20 : urls.py N+1 à N+6

Enfin des modifications ont été réalisées dans les fichiers HTML afin d'intégrer les nouveaux éléments d'affichage, les scripts JavaScript responsables des graphiques Highcharts ont été adaptés afin de permettre la visualisation dynamique des données provenant des différentes batteries. (A noté que chaque page HTML a un script différent pour correspondre aux données de la batterie)

```
<!-- Données JSON -->
<script type="application/json" id="data">
| | {{ data_json|safe }}
</script>

<!-- Script -->
<script>
document.addEventListener("DOMContentLoaded", function () {
    const dataElement = document.getElementById("data");
    if (!dataElement) return;

    let data = JSON.parse(dataElement.textContent);
    data.sort((a, b) => new Date(a.date) - new Date(b.date));

    const params = {
        total_volt: { label: "Tension (V)", container: "chart-voltage" },
        currentin: { label: "Courant Entrant (A)", container: "chart-currentin" },
        currentout: { label: "Courant Sortant (A)", container: "chart-currentout" },
        current_bat: { label: "Courant Batterie (A)", container: "chart-currentbat" }
    };
});
```

Figure 21 : visualisation\_graphique.html (partie script donnée)

## 8) Intégration MQTT (HiveMQ / TTN) dans Django

Au cours du projet, une partie du travail a porté sur l'analyse de l'architecture de communication basée sur le protocole MQTT.

Des travaux préalables avaient déjà été réalisés par les étudiants de la promotion précédente concernant l'utilisation de The Things Network (TTN) et du broker MQTT HiveMQ. Ces premiers développements avaient permis de valider le fonctionnement individuel de ces différentes solutions.

### 8.1) HiveMQ

HiveMQ est un broker MQTT permettant de gérer les échanges de messages entre différents équipements connectés. Son rôle principal est de recevoir les données publiées par les appareils, puis de les redistribuer aux applications abonnées à ces informations.

Dans le cadre du projet, HiveMQ est utilisé comme intermédiaire entre les équipements de collecte de données et l'application de supervision. Les informations transmises par les capteurs peuvent ainsi être centralisées avant d'être exploitées par les différents services du système.

L'utilisation d'un broker MQTT permet de faciliter les échanges de données tout en assurant une architecture évolutive adaptée aux projets IoT.

### 8.2) Intégration à Django

Pour l'intégration dans Django les anciens élèves avaient créé 2 fichiers pour la connexion entre HiveMQ et Django (TTN n'était pas retenu car la passerelle était instable) et fait des modifications dans le settings.py.



Le premier fichier start\_mqtt.py permet de démarrer le service MQTT

```

1
2 from django.core.management.base import BaseCommand
3 from GestionAdmin.services.mqtt_client import MQTTClient
4
5 class Command(BaseCommand):
6     help = 'Démarrage le client MQTT'
7
8     def handle(self, *args, **options):
9         self.stdout.write(self.style.SUCCESS('Démarrage du client MQTT...'))
10        client = MQTTClient()
11        client.connect()
12
13        # Gardez le processus en vie
14        try:
15            while True:
16                pass
17        except KeyboardInterrupt:
18            client.disconnect()
19            self.stdout.write(self.style.SUCCESS('Client MQTT arrêté'))
20

```

Figure 22 : start\_mqtt.py

Le deuxième fichier mis en place est le mqtt\_client.py c'est le fichier principal censé récupérer les données du broker MQTT, il utilise notamment celery pour le traitement / traduction des données récupérées (sous forme de payload)

```

# Configuration du logger
logger = logging.getLogger(__name__)

# Tâche Celery pour traiter les messages MQTT
@shared_task
def process_mqtt_message(topic, payload_str):
    try:
        # Parse le message JSON
        payload = json.loads(payload_str)

        # Décode les données du capteur
        temperature, humidity = decode_sensor_data(payload['data'])

        if temperature is None or humidity is None:
            logger.error("Échec du décodage des données de température/humidité")
            return

        # Récupération des informations LoRaWAN
        rx_info = payload['rxInfo'][0] if payload['rxInfo'] else None
        tx_info = payload['txInfo'] if 'txInfo' in payload else None

        # Import ici pour éviter les imports circulaires
        from DB.models import SensorData

        # Enregistrement dans TimeScaleDB
        sensor_data = SensorData(
            timestamp=datetime.fromtimestamp(int(payload['timestamp'])),
            temperature=temperature,
            humidity=humidity,
            applicationName=payload['applicationName'],
            dev_eui=payload['devEUI'],
            device_name=payload['deviceName'],
            f_cnt=payload['fCnt'],
            rssi=rx_info['rssi'] if rx_info else 0,
            snr=rx_info['lorasnr'] if rx_info else 0,
            frequency=tx_info['frequency'] if tx_info else 0
        )
    except Exception as e:
        logger.error(f"Erreur lors du traitement du message MQTT: {e}")

```

Figure 24 : mqtt\_client.py tâche celery

```

class MQTTClient:
    def __init__(self):
        # Créer un nouveau client MQTT
        self.client = mqtt.Client()

        # Définir les callbacks
        self.client.on_connect = self.on_connect
        self.client.on_message = self.on_message

        # Configurer l'authentification
        if settings.MQTT_USERNAME and settings.MQTT_PASSWORD:
            self.client.username_pw_set(settings.MQTT_USERNAME, settings.MQTT_PASSWORD)

        # Configurer TLS/SSL si nécessaire
        if settings.MQTT_CA_CERT:
            self.client.tls_set(
                ca_certs=settings.MQTT_CA_CERT,
                certfile=settings.MQTT_CLIENT_CERT if hasattr(settings, 'MQTT_CLIENT_CERT') else None,
                keyfile=settings.MQTT_CLIENT_KEY if hasattr(settings, 'MQTT_CLIENT_KEY') else None,
                cert_reqs=ssl.CERT_REQUIRED if not settings.MQTT_TLS_INSECURE else ssl.CERT_NONE
            )

        # Désactiver la vérification du nom d'hôte si nécessaire
        if settings.MQTT_TLS_INSECURE:
            self.client.tls_insecure_set(True)

    def on_connect(self, self, client, userdata, flags, rc):
        logger.info(f"Connecté au Broker MQTT avec le code: {rc}")
        self.client.subscribe("application/1/#") # Topic MQTT

    def on_message(self, self, client, userdata, msg):
        """
        Délègue le traitement du message à une tâche Celery
        """
        try:
            # Envoyer le message à Celery pour traitement asynchrone

```

Figure 23 : Authentification



Le troisième fichier settings.py du projet principale (application principale) la configuration permet de créer la connexion entre le Broker MQTT (en l'occurrence HiveMQ) et le projet Django.

```
MQTT_BROKER_HOST = ''
MQTT_BROKER_PORT = 8883
MQTT_USERNAME = ''
MQTT_PASSWORD = ''
MQTT_TOPIC = ''
MQTT_KEEP_ALIVE= 60
MQTT_CA_CERT="/home/ubuntu/dev/monProjetDjango/cert/isrgrootx1.pem"
MQTT_TLS_INSECURE = False
```

Figure 25 : settings.py

### 8.3 Donnée batterie sur HiveMQ

Enfin ce travail d'analyse sur MQTT et plus particulièrement nous a permis de tester la récupération des données batterie avec le broker. Grâce à l'arrivée d'une nouvelle passerelle nous avons pu récupérer de manière et sans ordre précis les différentes données de la batterie (N+0 à N+7)

The screenshot displays the HiveMQ WebClient interface. On the left, the 'Connection Settings' section shows a green status bar indicating 'The WebClient is connected'. Below this, there are fields for 'Username' (bms\_user) and 'Password' (Monpassword123), along with a 'Disconnect' button. The 'Topic Subscriptions' section shows a list of topics with a 'QoS' of 0 and an 'Actions' column with a trash icon. On the right, the 'Messages' section shows a list of 64 messages. The messages are JSON objects containing various battery data points, such as 'total\_voltage\_V', 'current\_in\_A', 'current\_out\_A', 'current\_bat\_A', 'Energy\_today\_collected\_kWh', 'Energy\_today\_consummed\_kWh', 'Cell\_voltage\_min\_V', 'Cell\_voltage\_max\_V', 'Cell\_voltage\_bypass\_V', 'unused1', 'unused2', 'unused3', 'timestamp', 'tmp\_lowest\_deg', 'Low\_nr', 'tmp\_highest\_deg', 'High\_nr', 'Min\_charg\_temp\_deg', 'Min\_d is\_temp\_deg', 'Max\_temp\_deg', 'Energy\_stored\_kWh', 'Battery\_capacity\_kWh', 'SOC\_percent', 'unused', 'Cell\_voltage\_lowest\_V', 'Low\_nr', 'Cell\_voltage\_highest\_V', 'High\_nr', 'Sbyte 1', 'Sbyte2', 'Current\_Cell\_voltage\_V', 'Curr\_temp\_deg', 'Curr\_nr', 'Cell\_cnt', 'unused1', 'Isolation\_resistance\_kOhm', 'unused2', and 'timestamp'.

Figure 26 : HiveMQ qui récupère les données de la batterie

Après l'écriture de ce rapport nous continuerons à travailler sur la liaison entre HiveMQ et Django cela nécessitera la modification des codes présentés auparavant et permettra normalement de faire l'intégration des données de la batterie au site Web.

## 9) Visualisation Web

L'application web développée avec Django permet de centraliser et d'afficher les différentes données collectées par la centrale photovoltaïque. L'interface est accessible depuis un navigateur web et offre plusieurs pages dédiées à la consultation des informations.

La principale page de visualisation (tableau de bord) permet d'afficher les données issues de la batterie sous forme de graphiques interactifs. Ces graphiques sont générés à l'aide de la bibliothèque Highcharts téléchargée préalablement (voir 6) Modification Code HTML) et facilitent l'analyse de l'évolution des mesures au cours du temps.

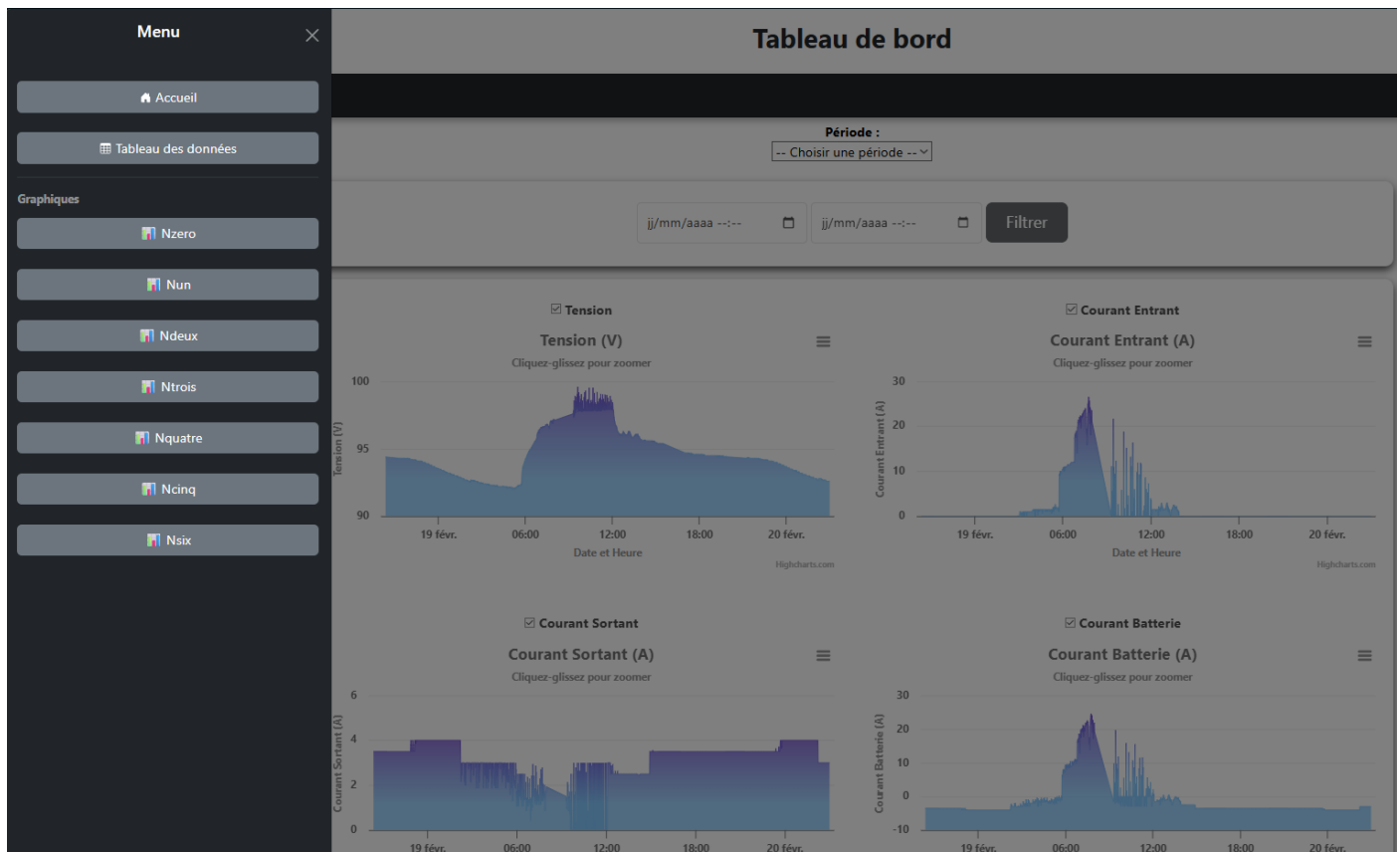


Figure 27 : Tableau de bord avec menu latérale

| Date                  | Tension (V) | Courant d'entrée (A) | Courant de sortie (A) | Courant de batterie (A) |
|-----------------------|-------------|----------------------|-----------------------|-------------------------|
| 19 février 2023 00:22 | 94,4        | 0,0                  | 3,5                   | -3,5                    |
| 19 février 2023 01:17 | 94,3        | 0,0                  | 3,5                   | -3,5                    |
| 19 février 2023 01:18 | 94,3        | 0,0                  | 3,5                   | -3,5                    |
| 19 février 2023 01:19 | 94,3        | 0,0                  | 3,5                   | -3,5                    |
| 19 février 2023 01:20 | 94,3        | 0,0                  | 3,5                   | -3,5                    |
| 19 février 2023 01:21 | 94,3        | 0,0                  | 3,5                   | -3,5                    |
| 19 février 2023 01:22 | 94,3        | 0,0                  | 3,5                   | -3,5                    |
| 19 février 2023 01:23 | 94,3        | 0,0                  | 3,5                   | -3,5                    |
| 19 février 2023 01:24 | 94,3        | 0,0                  | 3,5                   | -3,5                    |
| 19 février 2023 01:25 | 94,3        | 0,0                  | 3,5                   | -3,5                    |

« Première   Précédent   Page 1 sur 153   Suivant   Dernière »

Figure 28 : Tableau des données

## 10) PV de Recette

Ce cahier de recette a pour but de valider le travail principal effectué pendant le projet notamment les améliorations apportées, il permet de voir si le travail réalisé correspond aux attentes du cahier des charges.

| N° test | Description du test              | Résultat attendu                               | Résultat obtenu                                              | Conforme                                                             |
|---------|----------------------------------|------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------|
| 1       | Connexion SSH                    | Accès au serveur                               | Connexion réussie                                            | <input checked="" type="checkbox"/> Oui <input type="checkbox"/> Non |
| 2       | Restauration BDD locale          | Import de la sauvegarde                        | BDD accessible                                               | <input checked="" type="checkbox"/> Oui <input type="checkbox"/> Non |
| 3       | Chargement Highcharts local      | Affichage en statique sans CDN                 | Graphe affiché                                               | <input checked="" type="checkbox"/> Oui <input type="checkbox"/> Non |
| 4       | Ajout des données N+0 à N+6      | Afficher sur le site avec graphe correspondant | Le site affiche chaque donnée avec leur graphe correspondant | <input checked="" type="checkbox"/> Oui <input type="checkbox"/> Non |
| 5       | Connexion MQTT (HiveMQ) à Django | La connexion fonctionne avec envoi des données | La connexion n'a pas été établie                             | <input checked="" type="checkbox"/> Oui <input type="checkbox"/> Non |

## 11) Conclusion

Le projet SMACP m'a permis de participer à l'évolution et à la maintenance d'une application dédiée à la supervision d'une centrale photovoltaïque. Les différentes missions réalisées ont porté sur la prise en main de l'environnement existant, la récupération et la restauration d'une base de données PostgreSQL, la correction de plusieurs anomalies ainsi que l'intégration de nouvelles données au sein de l'interface web.

L'une des principales réalisations du projet a été l'intégration des données des batteries N+1 à N+6 dans les graphiques de visualisation. Ce travail a nécessité l'analyse du code existant ainsi que la modification de plusieurs composants de l'application Django, notamment les fichiers de routage, les vues, les modèles et les interfaces utilisateur.

Ce projet m'a également permis d'approfondir mes connaissances dans plusieurs domaines tels que l'administration de systèmes Linux, l'utilisation de PostgreSQL, le développement avec Django, la gestion de versions avec GitHub ainsi que l'utilisation de protocoles et technologies liés à l'Internet des Objets (IoT), notamment MQTT, HiveMQ et The Things Network.

Enfin ce projet peut être encore amélioré notamment au niveau de l'intégration de MQTT ou d'autres solutions pour la récupération des données ainsi que la mise à jour du site en temps réel.